
Control flow in Rust

`if`

`match`

`loop`

`while`

`for`



Gábor Takács

<dr.gabortakacs@gmail.com>

The five constructs

Construct	What it does	Its value
<code>if / else</code>	branches on a <code>bool</code>	the chosen branch
<code>match</code>	branches on <i>patterns</i>	the chosen arm
<code>loop</code>	repeats forever	<code>break</code> carries it out
<code>while</code>	repeats while a condition holds	<code>()</code>
<code>for</code>	walks through an iterator	<code>()</code>

All five are **expressions**, not statements: they have a value. For `while` and `for` that value is always the **unit value** `()`, so in practice only `if`, `match` and `loop` are useful on the right hand side of an `=`.

if: branching on a condition

```
if n % 2 == 0 { // no parentheses needed
    println!("even");
} else if n == 7 {
    println!("lucky");
} else {
    println!("odd");
}
```

- ▶ The condition **must be a bool**. There is no “truthy” value in Rust: `if 1 { }` is error[E0308]: expected `bool`, found integer.
- ▶ The braces are **mandatory**, even around a single statement.
- ▶ Parentheses around the condition are allowed but pointless, and rustc warns: *unnecessary parentheses around 'if' condition*.

if as an expression

```
let parity = if n % 2 == 0 { "even" } else { "odd" };
```

- ▶ This is what Python writes as `"even" if n % 2 == 0 else "odd"`; Rust needs **no separate syntax** for it.
- ▶ **Every branch must have the same type:**

```
let x = if n > 0 { 1 } else { "negative" };  
// error[E0308]: `if` and `else` have incompatible types
```

- ▶ **An else branch is required** when the value is used, otherwise there would be nothing to return in the false case:

```
let y = if n > 0 { 1 };  
// error[E0317]: `if` may be missing an `else` clause
```

match: branching on patterns

```
match n {  
  0          => println!("zero"),           // literal  
  1 | 2      => println!("one or two"),      // or-pattern  
  3..=9      => println!("three to nine"),   // range  
  x if x < 0 => println!("negative"),        // guard  
  _          => println!("everything else"),  
}
```

- ▶ The arms are tried **in order**; `_` matches anything.
- ▶ A **match must be exhaustive**: the compiler checks that every possible value is covered:

```
match n { 0 => println!("zero"), }  
// error[E0004]: non-exhaustive patterns:  
//   `i32::MIN..=-1_i32` and `1_i32..=i32::MAX` not covered
```

match as an expression

```
let description = match n % 3 {  
  0 => "a multiple of three",  
  1 => "one more than a multiple of three",  
  _ => "two more than a multiple of three",  
}; // note the closing semicolon
```

- ▶ Every arm must produce the **same type**.
- ▶ An arm may be a **block**; its value is the last expression:

```
let half_plus_one = match n {  
  0 => 0,  
  _ => {  
    let half = n / 2;  
    half + 1  
  }  
};
```

loop: the simplest loop

```
let mut i = 0;
loop {
    i += 1;
    if i % 2 == 0 { continue; } // next iteration
    println!("{i}");
    if i >= 7 { break; }        // leave the loop
}
```

- ▶ `loop` is Rust's infinite loop; it is preferred over `while true`, because the compiler `knows` it never finishes on its own.
- ▶ `break` leaves the loop, `continue` starts the next round.

break can carry a value

```
let mut i = 1;
let first_square_over_100 = loop {
    if i * i > 100 {
        break i * i; // this is the value of the loop
    }
    i += 1;
};
println!("{first_square_over_100}"); // 121
```

Only `loop` can do this. `while` and `for` may finish because their condition became false or the iterator ran out, and then there would be no value to hand back, so their `break` never takes an argument.

while: repeat while a condition holds

```
// Collatz: how many steps lead from 27 to 1?  
let mut n = 27;  
let mut steps = 0;  
while n != 1 {  
    n = if n % 2 == 0 { n / 2 } else { 3 * n + 1 };  
    steps += 1;  
}  
println!("{steps}"); // 111
```

- ▶ The condition must be a `bool` here as well.
- ▶ There is `no do ... while` in Rust;
it is written as a `loop` with the test at the end.

for: walking through a range

```
for i in 0..5 { print!("{i} "); } // 0 1 2 3 4
for i in 1..=5 { print!("{i} "); } // 1 2 3 4 5
for _ in 0..n { print!("{}",); }    // no variable needed
```

- ▶ `a..b` **excludes** the upper bound, `a..=b` **includes** it.
- ▶ The loop variable is **fresh in every iteration** and it is **not mutable**:
`for i in 0..3 { i += 1; }` is error[E0384].
- ▶ A `for` loop walks through *any* iterator, not only ranges.
Strings, arrays and collections come later.

Ranges: Python range vs. Rust

Python	Rust	Values
<code>range(5)</code>	<code>0..5</code>	0 1 2 3 4
<code>range(1, 6)</code>	<code>1..6</code> or <code>1..=5</code>	1 2 3 4 5
<code>range(0, 10, 3)</code>	<code>(0..10).step_by(3)</code>	0 3 6 9
<code>range(5, 0, -1)</code>	<code>(1..=5).rev()</code>	5 4 3 2 1

- ▶ `.rev()` and `.step_by()` are **iterator methods**, so the range needs parentheses around it.
- ▶ A negative step does not exist; reverse the range instead.

Nested loops: labels

```
// The first pair whose product is over 20.
'outer: for i in 1..10 {
    for j in 1..10 {
        if i * j > 20 {
            println!("{i} * {j} = {}", i * j);
            break 'outer; // break alone would only leave
                          // the inner loop
        }
    }
}
```

- ▶ A label starts with a **single quote**: 'outer:.
- ▶ It works with **loop**, **while** and **for**, and both **break 'label** and **continue 'label** are allowed.
- ▶ **break 'label value** is also possible, on a labelled **loop**.

Statements and expressions

```
let x = {  
    let a = 2;  
    let b = 3;  
    a * b           // no semicolon: the value of the block  
};                 // x == 6  
let y = {  
    let a = 2;  
    let _ = a * 3;  // semicolon: no value  
};                 // y == ()
```

- ▶ An **expression has a value**, a statement does not.
- ▶ A **block is an expression**: its value is the last expression inside it, **written without a semicolon**.
- ▶ `()` is Rust's "nothing": the **unit type** has exactly one value, also written `()`.

Typical beginner errors

- ☹️ `if 1 { }` (no truthy values) E0308
- ☹️ `let x = if c { 1 };` (the `else` is missing) E0317
- ☹️ `if c { 1 } else { "no" }` (branches of different types) E0308
- ☹️ a `match` that does not cover every value E0004
- ☹️ `for i in 0..3 { i += 1; }` (the loop variable is immutable) E0384
- ☹️ a stray semicolon after a block's last expression: the block becomes `()`
- 😊 the compiler names the error, points at the line and suggests a fix

Cheat sheet

```
if cond { .. } else if cond { .. } else { .. }  
let v = if cond { a } else { b };
```

```
match value {  
  0          => ..,      // literal  
  1 | 2      => ..,      // or-pattern  
  3..=9      => ..,      // range  
  x if x < 0 => ..,      // guard  
  -          => ..,      // everything else  
}
```

```
loop { .. break; .. continue; .. }  
let v = loop { .. break value; .. };  
while cond { .. }
```

```
for i in 0..n      { .. } // 0, 1, .., n-1  
for i in 0..=n     { .. } // 0, 1, .., n  
for i in (0..n).rev() { .. }  
for i in (0..n).step_by(k) { .. }  
'outer: for .. { for .. { break 'outer; } }
```